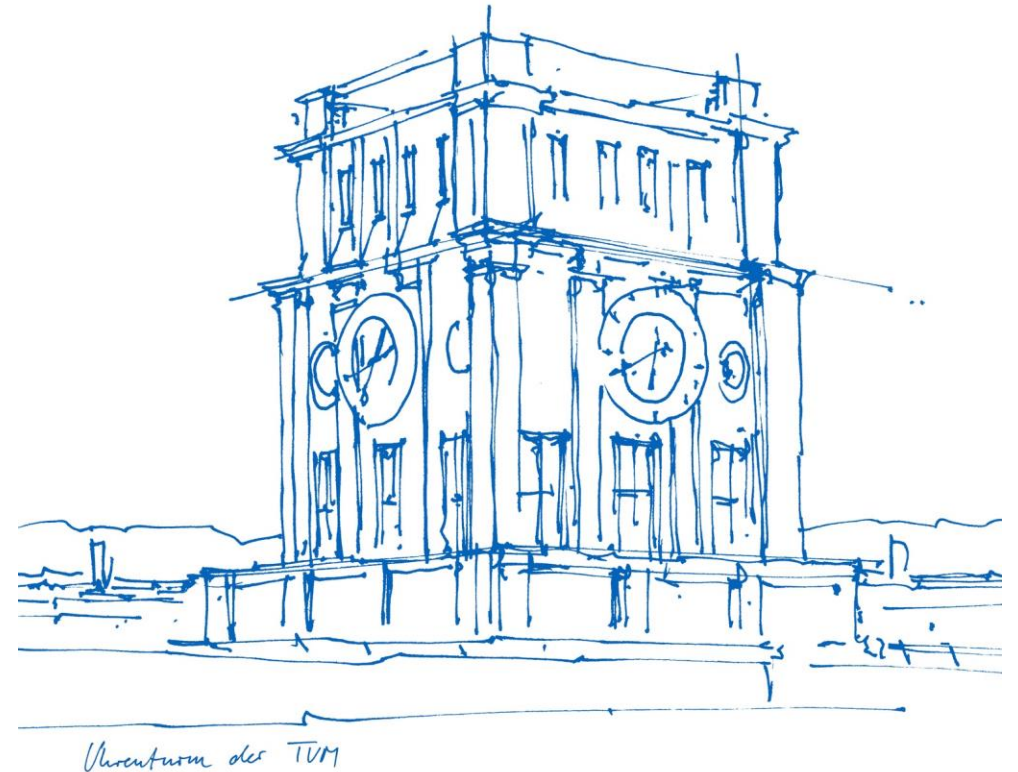


Memory Cache Simulation in SystemC

Lehrstuhl für Design Automation

GRA Abschlussprojekt – t034

Julian Reisch, Manar Ajir

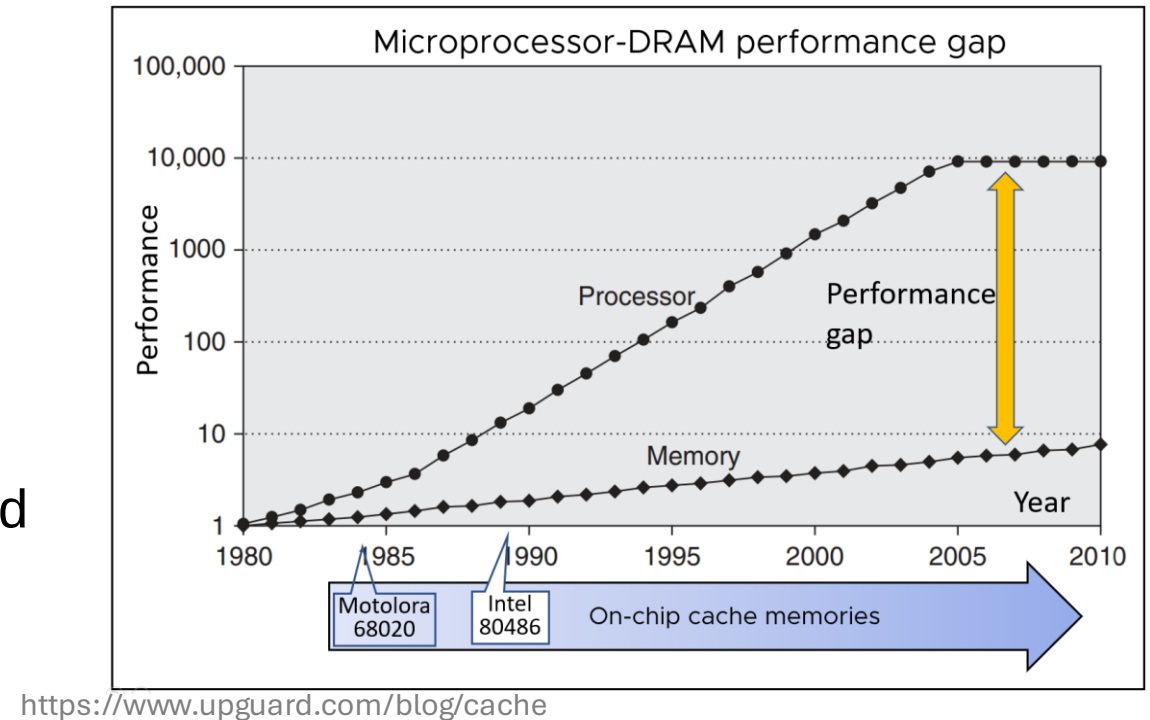


Inhaltsangabe

- Einführung
- Aufgabenstellung
- Implementierung
- Analyse der Implementierung
- Speicherzugriffsverhalten der Matrixmultiplikation
- Korrektheit
- Ausblick
- Quellen

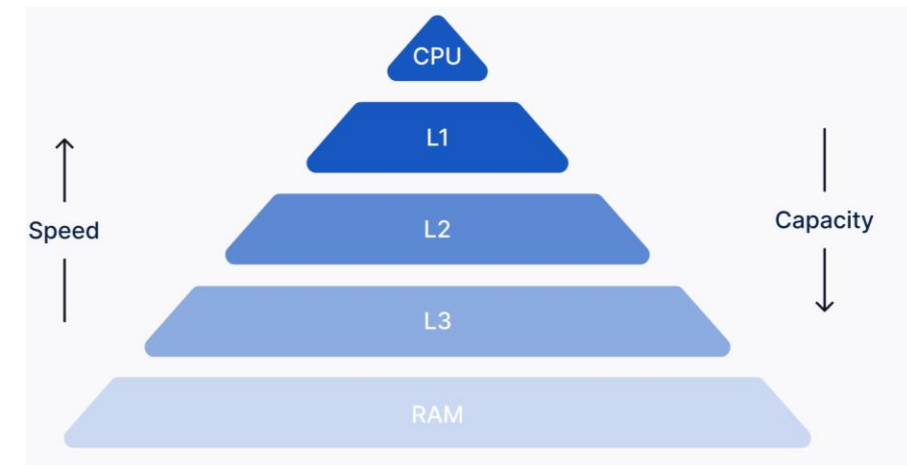
Einführung: Der von Neumann-Flaschenhals

- Prozessoren werden schneller besser als Speicher
- Prozesse müssen vergleichsweise länger auf Daten warten
- Führt zu Leistungseinbußen im gesamten System
- Verbesserung: Cache zwischen Prozessor und RAM
- Im Cache werden Daten in Cachezeilen gespeichert (z.B. 64 Byte)



Einführung: Speicherhierarchie und Cache-Management

- Moderne Caches bestehen aus mehreren **Cache-Levels** (L1, L2, L3)
- Niedrigere Ebenen sind größer aber auch langsamer
- L3 wird meistens von allen Kernen geteilt



Cache-type	Größe	Latenz
L1	16 – 128 KB	~4 cycles
L2	256 KB – 2 MB	~12 cycles
L3	2 – 64 MB	~40 cycles

Einführung: Speicheradressen

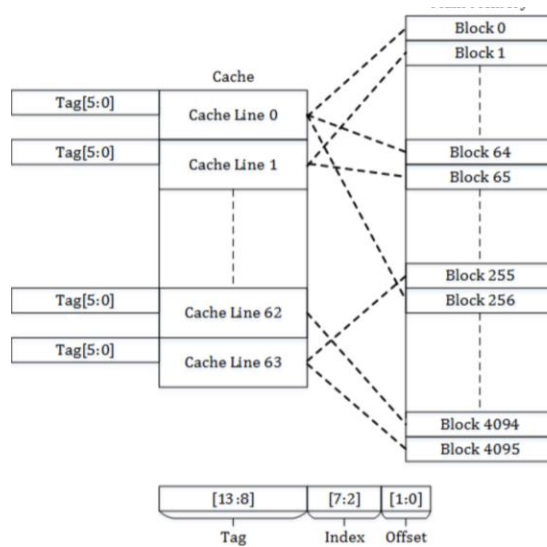
- Adresse wird in Tag, Index und Offset aufgeteilt
- **Tag**: identifiziert eine Zeile im Cache
- **Index**: wird verwendet, um zu entscheiden, wo eine Cachezeile gespeichert wird
- **Offset**: gibt das Byte innerhalb einer Cachezeile an, ab dem gelesen / geschrieben werden soll

→ Die Aufteilung der Speicheradresse hängt von der Mapping-Strategie ab

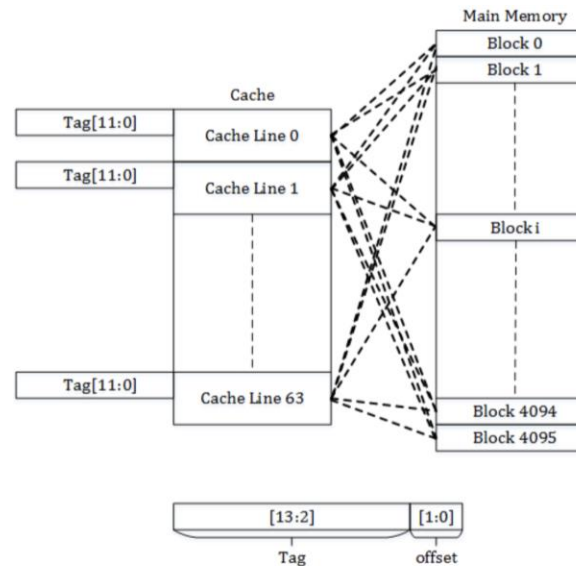


Einführung: Mapping Strategien

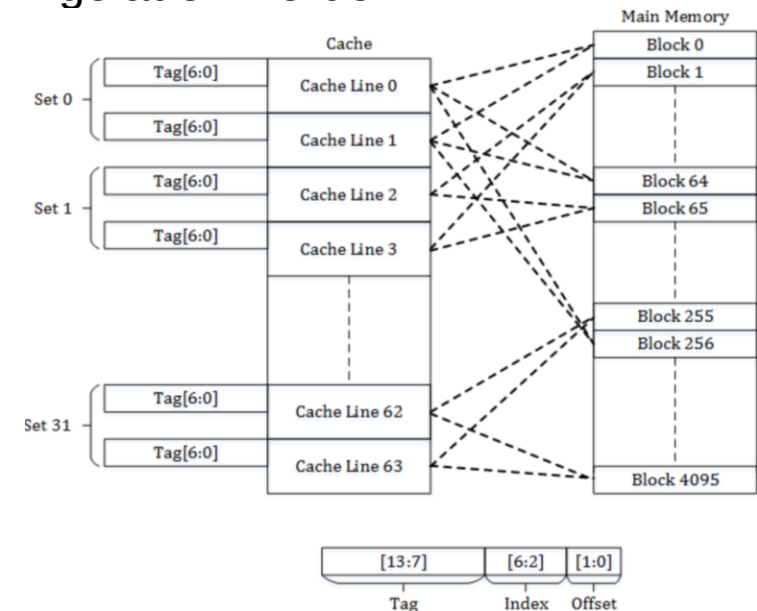
Direct-Mapped Cache: Jeder Speicherblock kann nur in einer Cachezeile stehen.



Fully-Associative Cache: Jeder Speicherblock kann in jede beliebige Cachezeile geladen werden.



Set-Associative Cache: Cache ist in Sets aufgeteilt, Speicherblock kann in eine beliebige Zeile eines Sets geladen werden.



Einführung: Ersetzungsstrategien im Cache

- **Least Recently Used (LRU):** Verdrängt die am längsten nicht genutzte Zeile, aufwendig, aber lokalitätsfreundlich.
- **First-In First-Out (FIFO):** Einfach, aber verdrängt auch kürzlich genutzte Daten.
- **Least Frequently Used (LFU):** Nutzt Zugriffszähler, gut für gleich bleibende Relevanz (z.B. Datenbanken).
- **Random Replacement:** Zufällige Ersetzung, einfach, aber ineffizient bei Lokalität. Wird teilweise in L3-Caches verwendet.

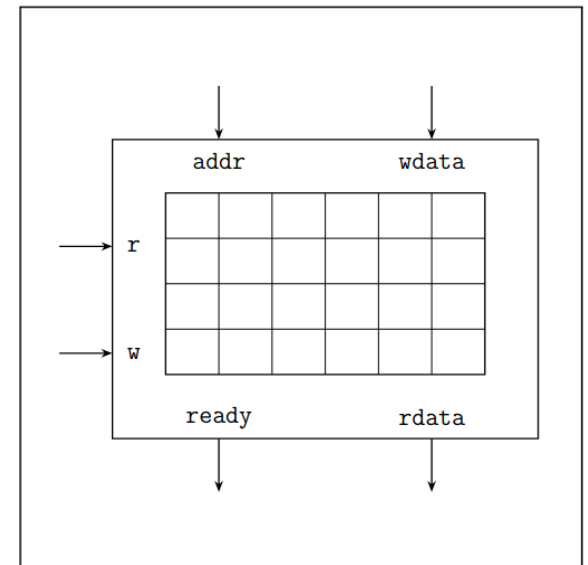
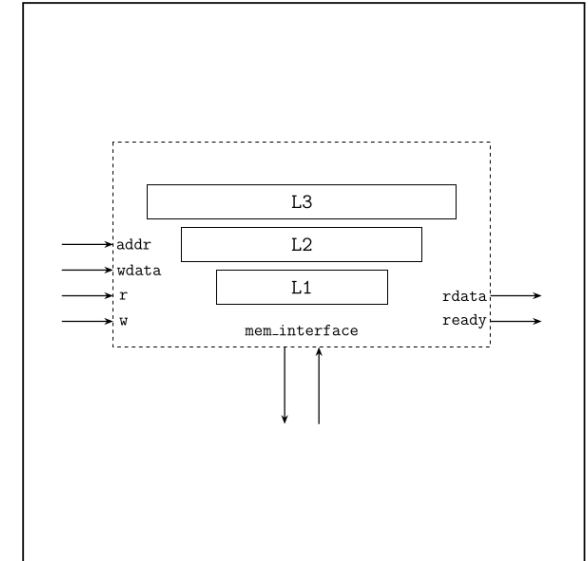
→ LRU ist am verbreitetsten

Aufgabenstellung

- Implementierung eines mehrstufigen Cache-Modells (L1, L2, L3)
- Untersuchung der drei Mapping-Strategien: direct-mapped, fully associative, set-associative
- Inklusiver Cache: Alle Daten im L1-Cache sind auch im L2- und L3-Cache vorhanden, um Konsistenz zu gewährleisten.
- Write-through: Daten werden sowohl in den Cache als auch in den Hauptspeicher geschrieben.
- Replacement-Strategie durfte frei gewählt werden → LRU da sehr verbreitet
- Untersuchen eines speicherintensiven Algorithmus

Implementierung

- CACHE-Modul: Verarbeitet Lese- und Schreibrequests indem es sie an die einzelnen Level bzw das RAM weiterleitet.
- L1, L2 und L3 sind Instanzen des LEVEL-Moduls (Wiederverwendbarkeit)
- MAIN_MEMORY-Modul aus der Hausaufgabe
- Cache Sets und Cachezeilen sind C++ Klassen
- Die Mapping-Strategien erfordern keine Fallunterscheidungen
 - direct-mapped und fully associative sind Sonderfälle des set-associative Caches
 - **Direct-mapped**: eine Cachezeile pro Set.
 - **Fully associative**: ein einziges Set für alle Cachezeilen.



Implementierung

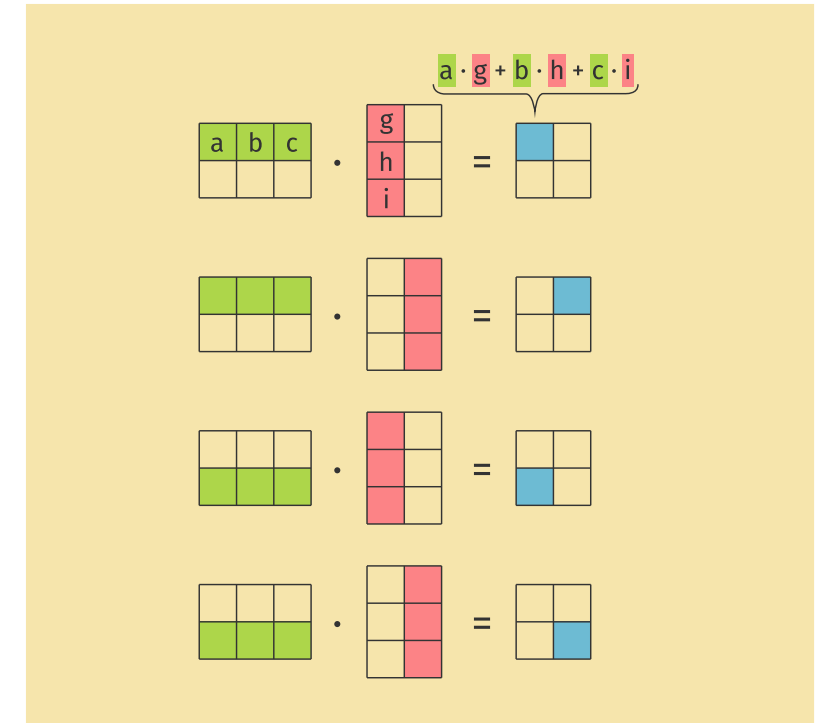
- Cache-Sets werden in einer `std::unordered_map<uint32_t, CacheSet>` gespeichert und über den index adressiert.
- Cachezeilen sind in einer doppelt verketteten Liste gespeichert, um Least Recently Used (LRU) effizient umzusetzen.
- Eine weitere unordered map pro Cache-Set ermöglicht den direkten Zugriff auf die Cachezeilen durch das Mapping von Tags auf Listen-Iteratoren.

Analyse der Implementierung

- Speicherverbrauch
 - Es werden nur Cachezeilen gespeichert, die beschrieben wurden
 - Cachezeilen werden komplett mit Nullen initialisiert → nicht geeignet bei sehr großen Cachezeilen
- Performanz
 - Die Simulation kann etwa 60.000 Cache-Anfragen pro Sekunde simulieren
 - Lese- und Schreib-Zugriffe können in amortisiert konstanter Zeit (unabhängig von der Cache-Größe) simuliert werden, da unordered maps genutzt werden.
- Code Duplikate
 - Wurden überwiegend vermieden
 - z.B. keine Fallunterscheidung für die mapping Strategien

Speicherzugriffsverhalten der Matrixmultiplikation

- **Methode:** Naive Matrixmultiplikation in Python programmieren, Lese- und Schreibzugriffe werden in .csv Datei geloggt.
- **Optimierung:** Es wird angenommen, dass das Zwischenergebnis des Skalarprodukts von Zeile und Spalte in einem CPU-Register liegt.
- **Ziel:** Einfluss der Mapping-Strategie und der Anzahl der Cache-Level auf Cache Misses untersuchen
- Es wurden konstante Werte für Cachezeilengröße, Level-Größen und Latenzen gewählt.



Speicherzugriffsverhalten der Matrixmultiplikation

Verfahren	Hits	Misses	Cycles
direct-mapped (1 Level)	30656	35904	31M
fully associative (1 Level)	32640	33920	30M
set-associative (1 Level)	31552	35008	31M
set-associative (2 Level)	55896	10664	11M
set-associative (3 Level)	66176	384	2M

- set-associative Caches haben weniger Misses als direct-mapped, aber mehr Misses als fully associative Caches
- Bei 3 Levels wurde die gesamte Matrix in den Cache geladen, sodass nur beim ersten Zugriff Misses auftraten.
- **Problem:** Langsame Zugriffe auf Spalten in B (nicht benachbart im Speicher)
- **Lösung:** Transposition von B vor Multiplikation

Korrektheit

Grundansätze zur Korrektheitsprüfung:

- *Verifikation*: Vollständige Verifikation aufgrund der SystemC-Komplexität und Parallelität schwer umsetzbar.
- *Tests*: können Korrektheit nicht garantieren, aber mögliche Eingaben und Fälle abdecken

Methodik:

- Base Case Testing: normale Fälle und Eingaben prüfen
 - Aligned Zugriffe: Alle vier Cache Rückgaben (Read Hit, Read Miss, Write Hit, Write Miss) prüfen
 - Räumliche Lokalität: prüfen, dass nach einem Miss die umgebenden Adressen einen Hit erzeugen
 - Zeitliche Lokalität: der Zeitpunkt der Zugriffe muss richtig geloggt werden (Clock ausgeben oder Tracefile überprüfen)

Korrektheit

- Edge Case Testing:
 - Testen, ob nicht-aligned Zugriffe (zum Beispiel ungerade Adressen) ein richtiges Ergebnis liefern
 - Testen, ob Zugriffe über Cachezeilen hinaus den Cache und das RAM richtig modifizieren
 - Verhalten bei Capacity Misses überprüfen: Wird die richtige Cachezeile verdrängt?
 - Mehr-Level-Nutzung: Inklusivität gewährleistet?
- *Automatisierte Tests*: --test Option ermöglicht testen, Programmierfehler können frühzeitig erkannt werden

Ausblick

- Mögliche Verbesserungen:
 - Bei write Cache Misses wird nach dem Schreiben immer die ganze Zeile geladen, obwohl 4 Byte schon bekannt sind.
 - andere Modulstruktur: Levels direkt miteinander verbinden statt indirekter Verbindung über Cache-Modul
 - Wenn auf viele Cache Lines nur einmal zugegriffen wird, belegt die Simulation unnötig viel Speicher.
 - Latenzen sind nicht perfekt, einzelne Taktzyklen Wartezeit um sicherzustellen, dass Signale verarbeitet wurden
 - Umfangreichere Tests, um selten auftretende Fehler zu finden

Quellen:

- [ERA Vorlesungsfolien W23/24](#)
- <https://www.upguard.com/blog/cache> [aufgerufen am 11.01.25]
- <https://dev.to/larapulse/cpu-cache-basics-57ej> [aufgerufen am 11.01.25]
- <https://www.includehelp.com/operating-systems/cache-memory-and-its-different-levels.aspx> [aufgerufen am 18.01.25]
- https://en.wikipedia.org/wiki/Cache_placement_policies [aufgerufen am 27.01.25]
- <https://hazelcast.com/foundations/caching/caching-strategies> [aufgerufen am 29.01.25]
- <https://library.fiveable.me/advanced-computer-architecture/unit-8/multi-level-cache-hierarchies/study-guide/DFgwyEGiVkJgHEe> [aufgerufen am 30.01.25]
- <https://www.geeksforgeeks.org/cache-memory-in-computer-organization/> [aufgerufen am 30.01.25]